

Replace Pi

"pi" → "3.14"

Replace Pi

Given a string, represented as a character array, design a recursive algorithm to **replace** all the **pi**'s in the given string with **3.14** & print the resulting string.

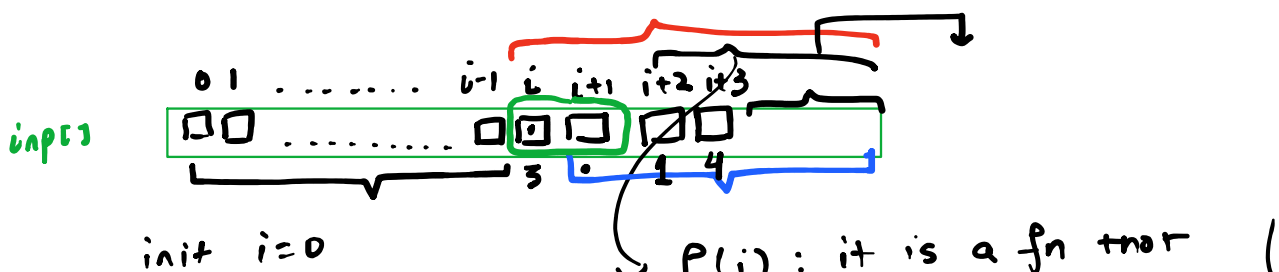
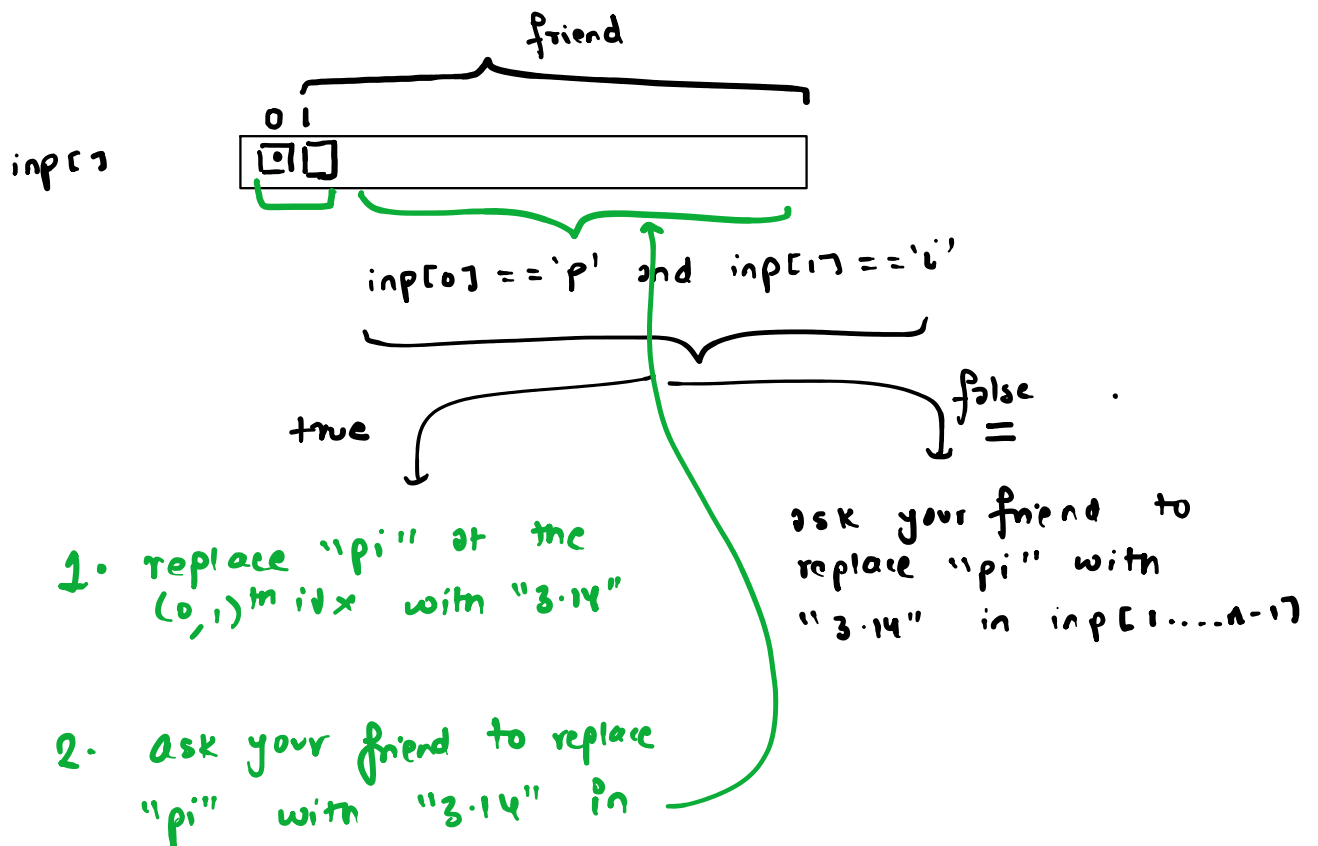
Example

Input : "pip"
Output : "3.14p"

Input : "pippi"
Output : "3.14p3.14"

Handwritten examples of string replacement:

- Input: "pip" → Output: "3.14p"
- Input: "pippi" → Output: "3.14p3.14"
- Input: "pipipi" → Output: "3.14i3.14"
- Input: "pipipi" → Output: "3.14i3.14"



init $i=0$
 $f(0)$

$f(i)$: it is a fn that
replaces all the
"pi" with "3.14"
in $inp[i \dots n-1]$

current
problem

$inp[i] == 'p'$ and $inp[i+1] == 'i'$

true

false

1. replace "pi" at
the $(i, i+1)$ th idx
with "3.14"

$f(i+1)$

a) shift all char. starting
from idx $i+2$ 2-steps
fwd

b) $\left\{ \begin{array}{l} inp[i] = '3'; \\ inp[i+1] = '.'; \\ inp[i+2] = '1'; \\ inp[i+3] = '4'; \end{array} \right.$

✓
✓

2. $f(i+4)$

↳ Base Case will be in terms of i

$\left\{ \begin{array}{l} \text{if } (inp[i] == '\backslash 0') \{ \\ \quad \text{ret}; \\ \} \end{array} \right.$

Generate Subsequences

↳ for a seq. of size n , there will be 2^n subsequences

Generate Subsequences

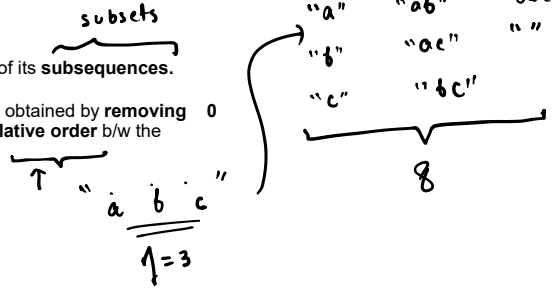
Given a string, design a recursive algorithm to generate all of its subsequences.

A **subsequence** of a sequence is a sequence which can be obtained by removing 0 or more elements from the given sequence such that the **relative order** b/w the sequence elements **doesn't change**.

Example

Input : "abc"

Output : "abc", "ab", "ac", "a", "bc", "b", "c", ""



∴ P
"a c b d"
"c a b" x

" a b c "

2 . 2 . 2 = 2^3

✓ x ✓ = ac

✓ ✓ x = ab

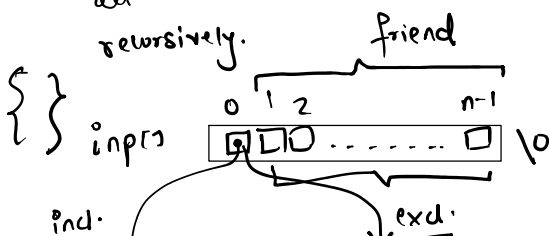
x x ✓ = c

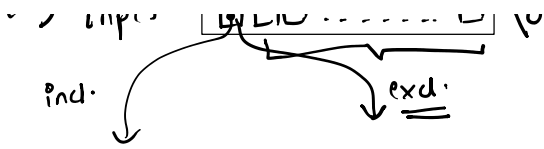
✓ ✓ ✓ = abc

⋮

→ Whenever while solving a question, you've to take a seq. of decisions then to build a recursive solⁿ, all you've to do is take the 1st decision, rest all the decisions will done / taken recursively (friend)

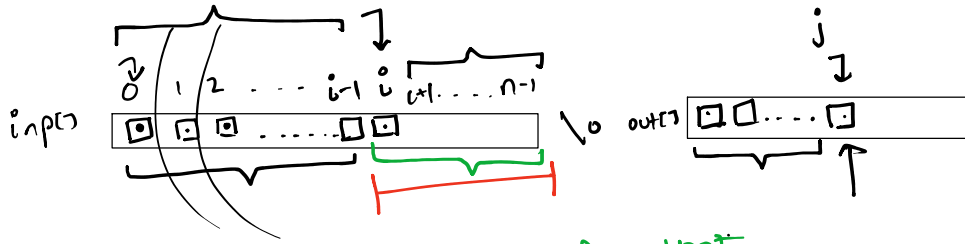
→ To build a subseq we've to take a seq. of n -decisions — a decision for each char. whether to incl. it or excl. it from o/p. ∴ to build a recursive solⁿ to solve this problem all we've to do is decide for the 1st char, rest all the decisions can be taken recursively. friend





{inp₀}

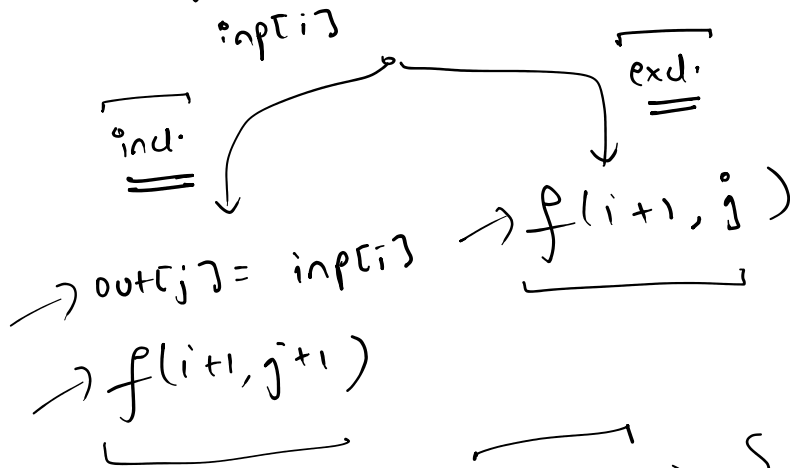
{ }



$\Rightarrow f(i, j)$ = it is fn that takes decisions for char. in $inp[i \dots n-1]$ s.t next free idx in $out[]$ is j .

init
 $i = 0$
 $j = 0$

to solve this ques.
just decide for



if ($i == n$) {
 $out[j] = '\0'$;
 $cout << out$;
 $\rightarrow ret$;
}

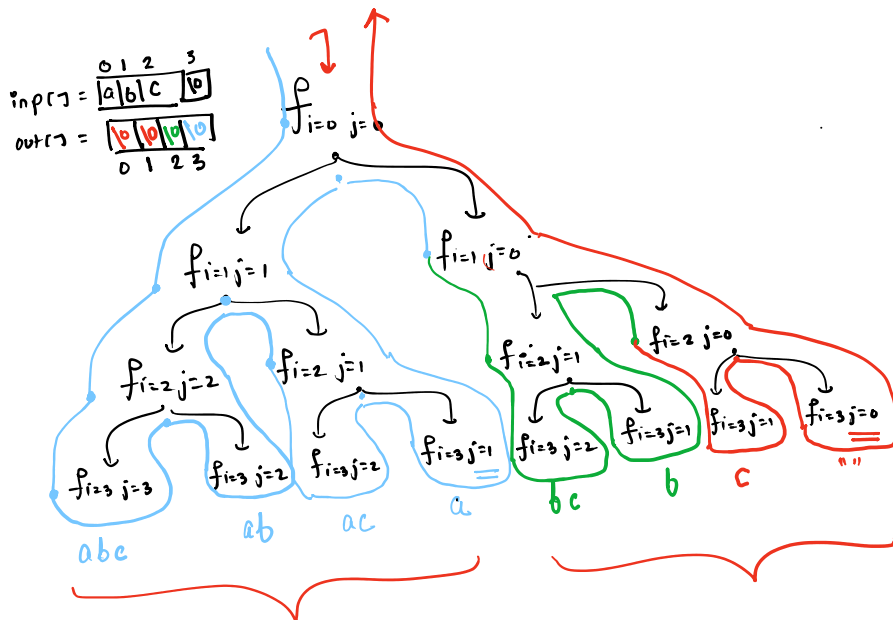
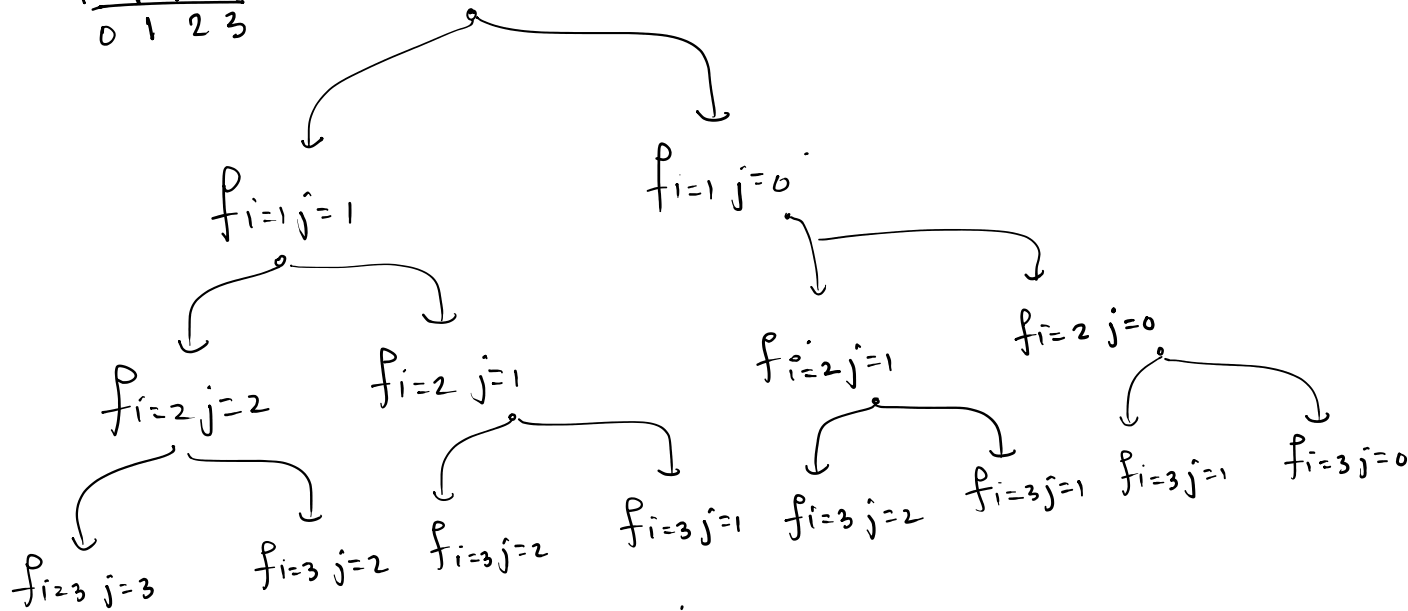
$inp[] = \begin{matrix} 0 & 1 & 2 & 3 \\ a & b & c & \backslash 0 \end{matrix}$
 $out[] = \begin{matrix} 0 & 1 & 2 & 3 \end{matrix}$

$f_{i=0, j=0}$

out[] =

0	1	2	3

$i=0, j=0$



Generate Parentheses

Generate Parentheses

Given **N** pairs of parentheses, design a recursive algorithm to **generate** all the combinations of parentheses.

Example

Input : $N = 2$

Output : "()", "()()", ")()(", ...

$n=2$

(.)
()
· (())
· ()()
·))((
·)(())
· (())()
·)()()

$n=3$

()
()
()
⏟

((()))

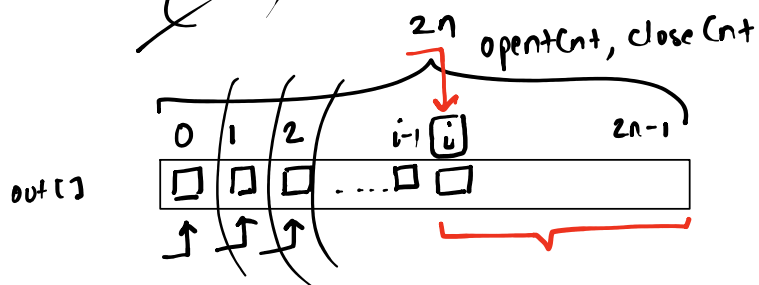
() () ()

(()) ()

⋮

$n=4$

~~()~~ ~~()~~ () () () ()
~~()~~ ~~()~~ () () () ()
~~()~~ () () () ()
~~()~~ () () () ()



$f(i, oc, cc) =$ take decisions for remaining pos. i to $2n-1$

init $i=0$

$oc=0$

$cc=0$

to solve this just decide for the

to solve this we
decide for the

openCnt < n

at pos

closeCnt < n

'('

)

out[i] = '('

f(i+1, oc+1, cc)

out[i] = ')'

f(i+1, oc, cc+1)

if (i == 2n) {

out[i] = '\0';

cout << out;

ret;

}

oc == n and cc == n

Generate Balanced Parentheses

Generate Balanced Parentheses

Given **N** pairs of parentheses, design a recursive algorithm to **generate** all the combinations of well-formed parentheses.

Example

Input : $N = 2$
Output : "(())", "()()"

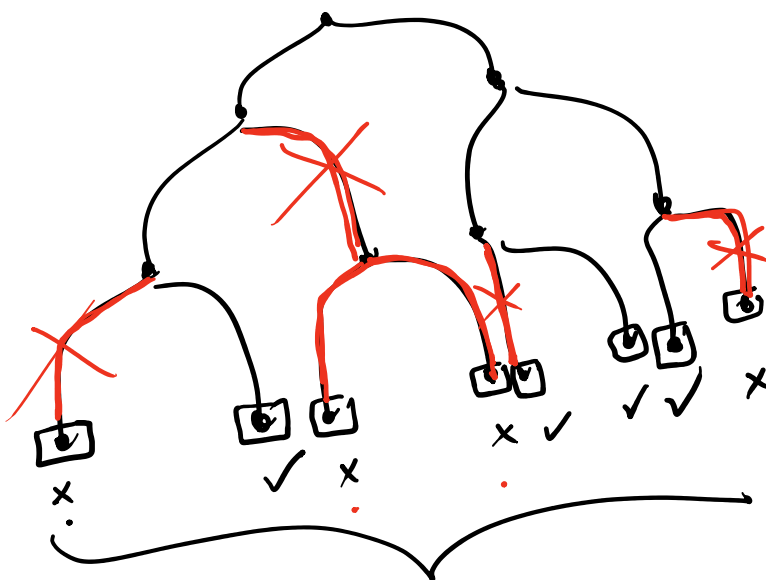
Input : $N = 3$
Output : "((()))", "(()())", "(())()", "()(())", "()()()"

$n = 2$ (())
 () ()

)) ((X
) (() X

((()))
() () ()
(()) ()
() (())
(() ())

())) ((X



Coin Toss

Given **N** coins, such that each coin has two sides **H** (heads) and **T** (tails), design a recursive algorithm to **generate** all the possible **outcomes** that we can get upon tossing the N coins simultaneously.

Example

Input : N = 2

Output : "HH", "HT", "TH", "TT"

Input : N = 3

Output : "HHH", "HHT", "HTH", "HTT", "THH", "THT", "TTH", "TTT"

Restricted Coin Toss

Given **N** coins, such that each coin has two sides **H** (heads) & **T** (tails), design a recursive algorithm to generate all the possible **outcomes** that we can get upon tossing the N coins **simultaneously** such that an outcome **should not** contain **consecutive heads**.

Example

Input : N = 2
Output : "HT", "TH", "TT"

Input : N = 3
Output : "HTH", "HTT", "THT", "TTH", "TTT"

[HW] Creating Expression

<https://codeforces.com/group/MWSDmqGsZm/contest/223339/problem/V>

[HW] Generate Binary Strings without Adjacent Zeros

<https://leetcode.com/problems/generate-binary-strings-without-adjacent-zeros/description/>